

Experiment No. 6

Implementation and Comparative Analysis of Image Segmentation Techniques using Python

and OpenCV

```
In [2]: import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Step 1: Load image
image = cv2.imread("image.jpg.png")
if image is None:
    print("Image not found!")
    exit()

# Step 2: Grayscale and Gaussian Blur
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Step 3: Global Thresholding
_, global_threshold = cv2.threshold(blur, 127, 255, cv2.THRESH_BINARY)

# Step 4: Otsu's Thresholding
otsu_value, otsu_threshold = cv2.threshold(
    blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
print("Otsu Threshold:", otsu_value)

# Step 5: Adaptive Thresholding
adaptive_threshold = cv2.adaptiveThreshold(
    blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 11, 2
)

# Step 6: Watershed Segmentation
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(otsu_threshold, cv2.MORPH_OPEN, kernel, iterations=2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)

dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.7 * dist.max(), 255, 0)
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

_, markers = cv2.connectedComponents(sure_fg)
markers += 1
markers[unknown == 255] = 0

watershed_image = image.copy()
markers = cv2.watershed(watershed_image, markers)
watershed_image[markers == -1] = [255, 0, 0]

# Step 7: K-Means Clustering
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```

pixels = np.float32(rgb.reshape((-1, 3)))

kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
labels = kmeans.fit_predict(pixels)
centers = np.uint8(kmeans.cluster_centers_)

segmented_image = centers[labels].reshape(rgb.shape)

# Step 8: Compare Results
plt.figure(figsize=(12, 8))

images = [
    (rgb, "Original Image"),
    (global_threshold, "Global Thresholding"),
    (otsu_threshold, "Otsu Thresholding"),
    (adaptive_threshold, "Adaptive Thresholding"),
    (cv2.cvtColor(watershed_image, cv2.COLOR_BGR2RGB), "Watershed"),
    (segmented_image, "K-Means")
]

for i, (img, title) in enumerate(images, 1):
    plt.subplot(2, 3, i)
    plt.imshow(img, cmap="gray")
    plt.title(title)
    plt.axis("off")

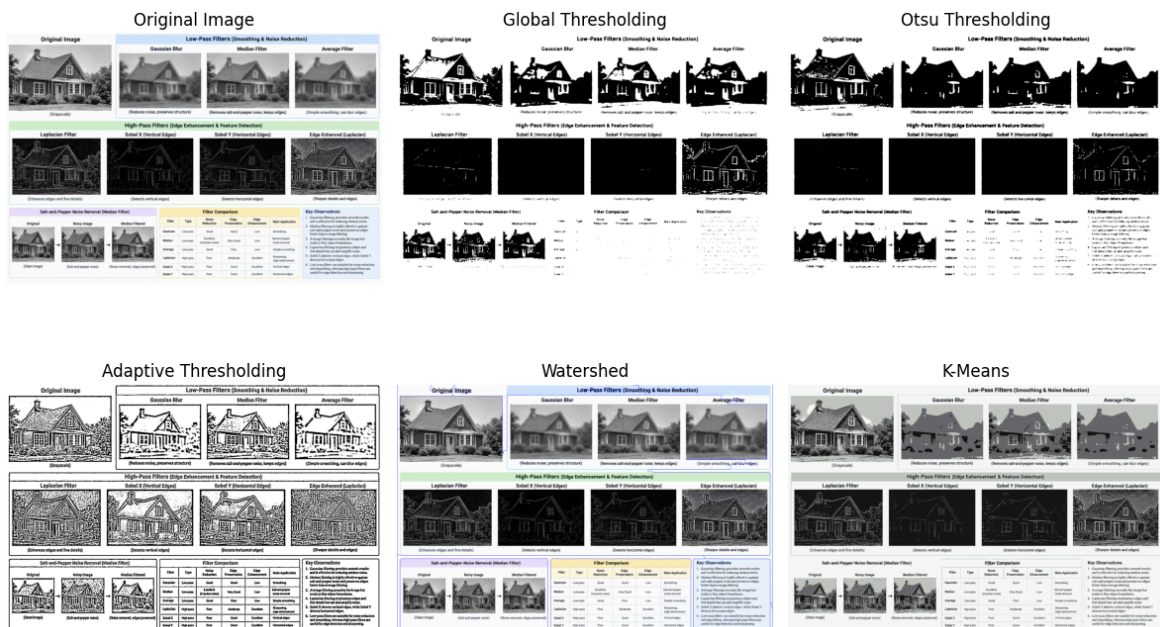
plt.tight_layout()
plt.show()

# Step 9: Observations
print("\nObservations:")
print("Global: Simple and fast")
print("Otsu: Automatic threshold")
print("Adaptive: Handles uneven illumination")
print("Watershed: Separates touching objects")
print("K-Means: Color-based segmentation")

# Step 10: Save Results
cv2.imwrite("global_threshold.jpg", global_threshold)
cv2.imwrite("otsu_threshold.jpg", otsu_threshold)
cv2.imwrite("adaptive_threshold.jpg", adaptive_threshold)
cv2.imwrite("watershed.jpg", watershed_image)
cv2.imwrite("kmeans_segmentation.jpg",
            cv2.cvtColor(segmented_image, cv2.COLOR_RGB2BGR))

```

Otsu Threshold: 150.0



Observations:

Global: Simple and fast

Otsu: Automatic threshold

Adaptive: Handles uneven illumination

Watershed: Separates touching objects

K-Means: Color-based segmentation

Out[2]: True

Questions and Answers

1. What is image segmentation? Why is it considered a fundamental step in computer vision?

Image segmentation is the process of dividing an image into meaningful regions or segments based on properties such as color, intensity, texture, or boundaries. Each segment represents a specific object or part of an object.

It is a fundamental step because it helps separate important objects from the background and provides useful information for tasks such as object detection, recognition, medical image analysis, and autonomous driving.

2. Differentiate between Image Segmentation and Image Classification.

Image Segmentation	Image Classification
Divides an image into different regions or objects.	Assigns a label to the entire image.
Works at pixel or region level.	Works mainly at image level.
Identifies the location and shape of objects.	Identifies what the image represents.
Example: Separating a tumor from healthy tissue.	Example: Classifying an image as "tumor" or "normal".

3. Explain the working principle of Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

Global Thresholding:

A single threshold value is selected for the entire image. Pixels above the threshold are assigned to one class, while pixels below it are assigned to another class.

Otsu's Thresholding:

Otsu's method automatically calculates the best threshold by analyzing the image histogram. It selects a threshold that maximizes the separation between foreground and background.

Adaptive Thresholding:

Adaptive thresholding calculates different threshold values for different regions of an image. It is useful when the image has uneven or varying illumination.

4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?

The Watershed Algorithm is a segmentation technique that treats an image like a topographic surface. Bright and dark regions are considered different heights, and boundaries are created where different regions meet.

It is useful for separating overlapping objects because it can identify individual objects even when their boundaries touch or overlap. For example, it can separate connected cells in a medical image.

5. How is K-Means Clustering applied to image segmentation?

K-Means Clustering groups pixels into **K** clusters based on their similarity, usually using color or intensity values.

The basic steps are:

1. Select the number of clusters, **K**.
2. Assign initial cluster centers.
3. Assign each pixel to the nearest cluster center.
4. Recalculate the cluster centers.
5. Repeat the process until the clusters become stable.
6. Assign different regions to the resulting clusters.

This method can be used to divide an image into regions with similar colors or intensities.

6. Compare threshold-based segmentation and clustering-based segmentation techniques.

Threshold-Based Segmentation	Clustering-Based Segmentation
Uses one or more threshold values.	Groups pixels based on similarity.
Simple and fast.	Usually more computationally expensive.

Threshold-Based Segmentation	Clustering-Based Segmentation
Works well with clear foreground-background differences.	Can handle multiple regions and colors.
Sensitive to illumination changes.	More flexible for complex images.
Examples: Global, Otsu, Adaptive Thresholding.	Example: K-Means Clustering.

7. What challenges are encountered while segmenting images with complex backgrounds or varying illumination?

Some common challenges are:

- Uneven lighting across the image.
- Similar colors between objects and background.
- Shadows and reflections.
- Object boundaries that are unclear.
- Noise in the image.
- Overlapping or touching objects.
- Complex textures and backgrounds.

These problems can make it difficult to accurately separate foreground objects from the background.

8. Mention five real-world applications where image segmentation plays a critical role.

Five important applications are:

1. **Medical Imaging:** Detecting tumors, organs, and abnormal tissues.
2. **Autonomous Vehicles:** Separating roads, pedestrians, vehicles, and other objects.
3. **Satellite Imaging:** Identifying buildings, vegetation, roads, and water bodies.
4. **Agriculture:** Detecting crops, weeds, and plant diseases.
5. **Face and Object Recognition:** Separating objects or facial regions from the background.

9. How does image segmentation improve the performance of object detection and image recognition systems?

Image segmentation separates important objects or regions from irrelevant background areas. This reduces unnecessary information and allows the system to focus on the target object.

As a result, segmentation can improve object localization, feature extraction, recognition accuracy, and detection performance, especially when the background is complex.

10. Compare traditional image segmentation techniques with deep learning-based segmentation methods such as U-Net and Mask R-CNN.

Traditional Techniques	Deep Learning Techniques
Use manually designed rules or algorithms.	Learn features automatically from training data.
Examples include thresholding, watershed, and K-Means.	Examples include U-Net and Mask R-CNN.
Usually require less training data.	Generally require a large labeled dataset.
Easier to implement and understand.	More complex to train and implement.
Can struggle with complex images.	Usually performs better on complex real-world images.
Faster for simple segmentation tasks.	Can require more computational resources.

U-Net is widely used for pixel-level segmentation, especially in medical imaging. **Mask R-CNN** performs object detection along with instance segmentation, allowing it to identify and create a separate mask for each object.